



## Practical No 1

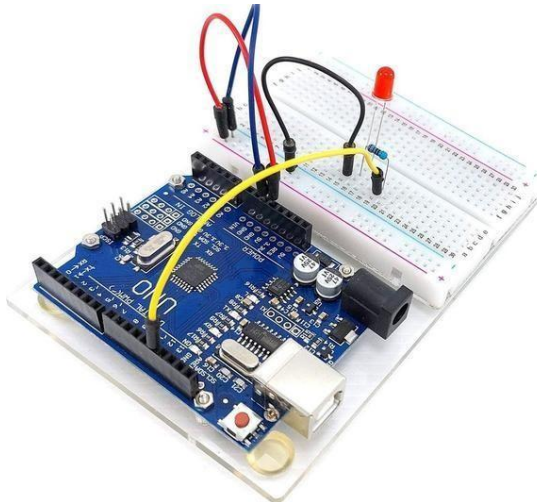
**Aim: To familiarize with the Arduino board and interface an LED/buzzer with Arduino to turn ON the LED for 1 second after every 2 seconds.**

### Theory

Arduino Uno is a microcontroller development board based on the ATmega328P. It provides digital and analog input/output pins that can be programmed using the Arduino IDE. An LED can be controlled by sending HIGH or LOW signals to a digital output pin. The delay() function is used to create time delays in milliseconds.

### Hardware Components

1. Arduino Uno
2. LED
3. 220Ω resistor
4. Breadboard
5. Jumper wires
6. USB cable
7. Buzzer — optional



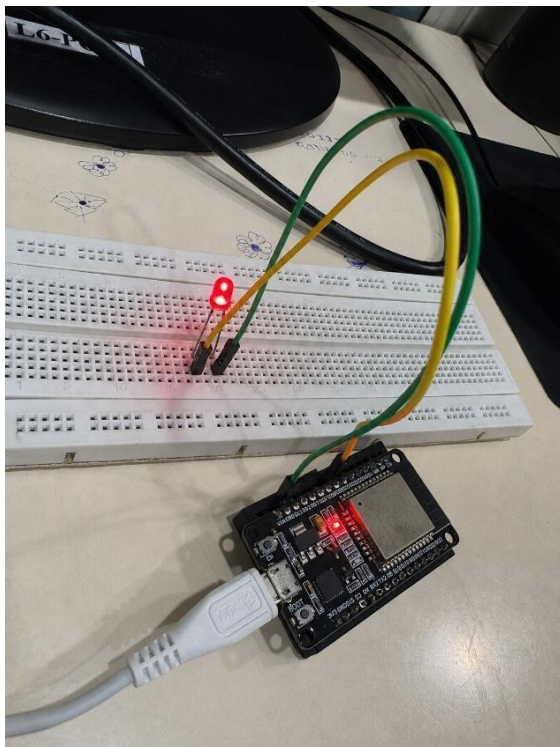
### Connections

| Arduino | Component                 |
|---------|---------------------------|
| D13     | LED through 220Ω resistor |
| GND     | LED cathode               |
| D8      | Buzzer +                  |
| GND     | Buzzer –                  |

### Code:

```
int ledPin = 13; void setup() {  
  pinMode(ledPin, OUTPUT);  
}  
void loop() {  
  digitalWrite(ledPin, HIGH);  
  delay(1000);  
  digitalWrite(ledPin, LOW);  
  delay(2000);  
}
```

### Output



### Conclusion

Thus, the Arduino was successfully programmed to control an LED with the required time delay.

**Output Photo 1:** Arduino connected to LED/buzzer.

**Output Photo 2:** LED in ON condition.

**Output Photo 3:** Arduino IDE showing successfully uploaded code.

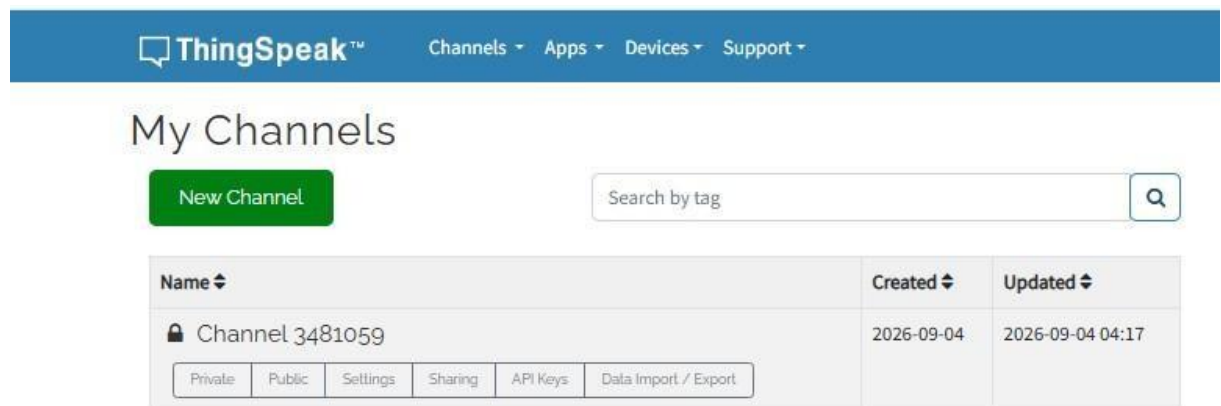
## Practical 2

**AIM:** To interface DHT11 with ESP8266 and display real-time temperature and humidity readings on the Serial Monitor. step:

Open: <https://thingspeak.mathworks.com/>

Login with MATLAB Online.

After Login:



ThingSpeak™ Channels ▾ Apps ▾ Devices ▾ Support ▾

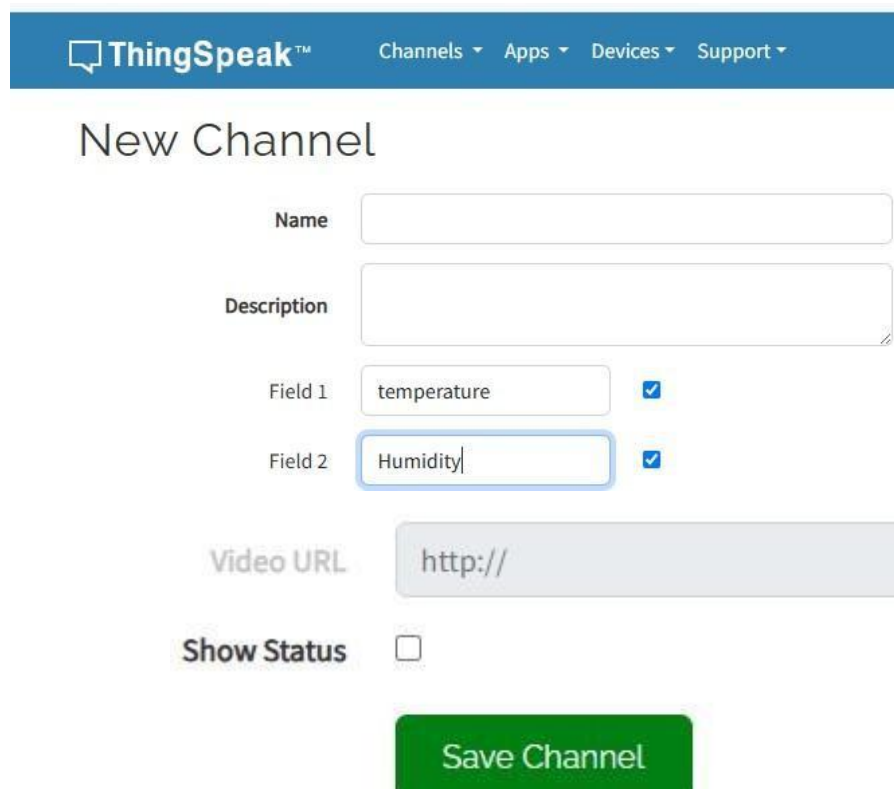
### My Channels

New Channel

Search by tag

| Name ▾          | Created ▾  | Updated ▾        |
|-----------------|------------|------------------|
| Channel 3481059 | 2026-09-04 | 2026-09-04 04:17 |

Private Public Settings Sharing API Keys Data Import / Export



ThingSpeak™ Channels ▾ Apps ▾ Devices ▾ Support ▾

### New Channel

Name

Description

Field 1  ☒

Field 2  ☒

Video URL

Show Status ☐

Save Channel

Click on API:

**Channel 3481059**  
Channel ID: 3481059  
Author: mwa0000038094008  
Access: Private

Private View Public View Channel Settings Sharing API Keys Data Import / Export

Add Visualizations Add Widgets Export recent data

MATLAB Analysis MATLAB Visualization

**Channel Stats**  
Created: less than a minute ago

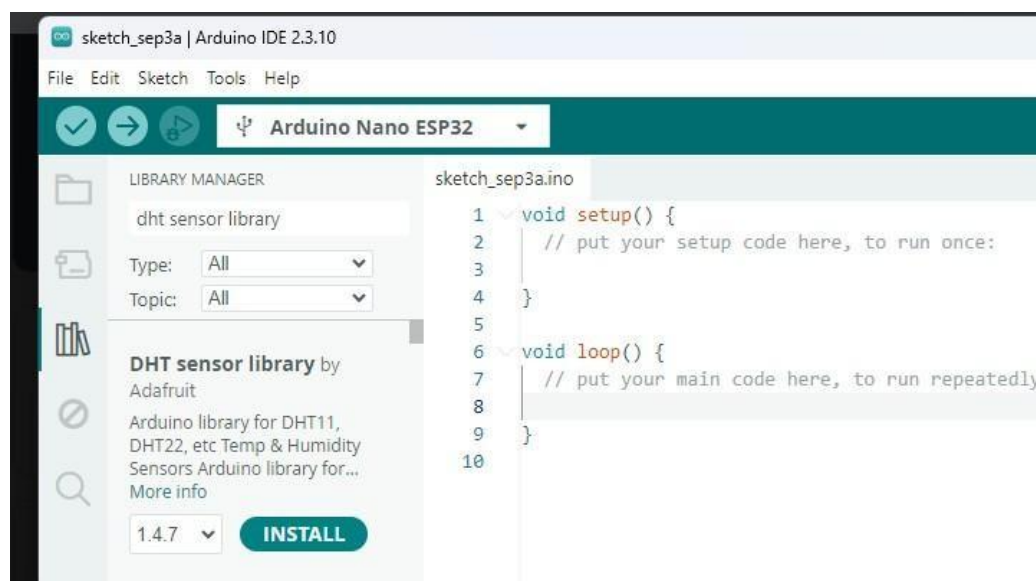
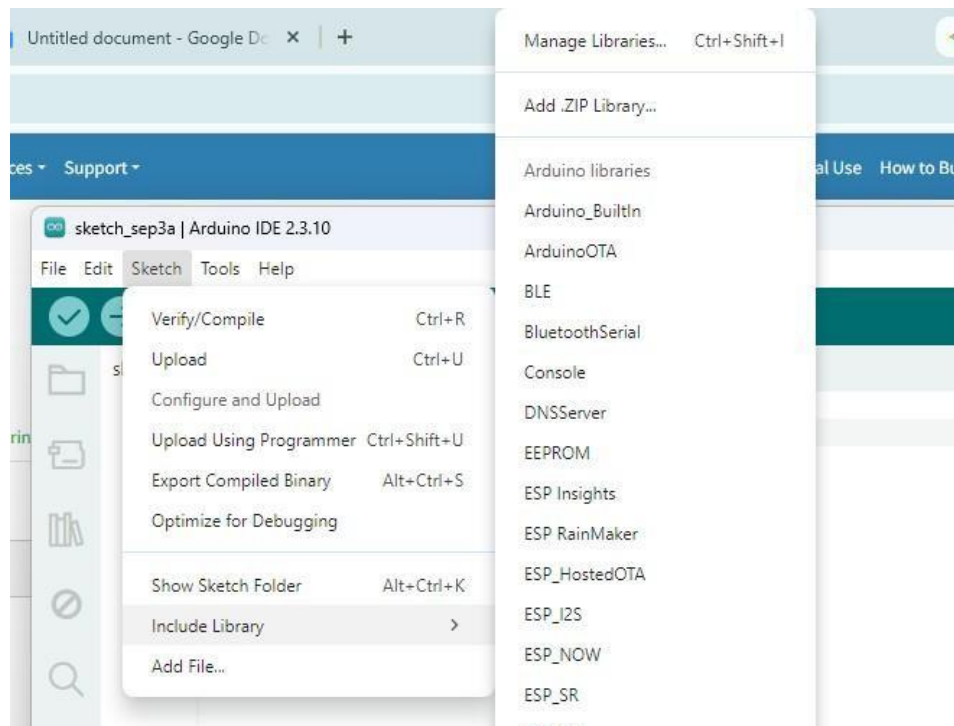
**Write API Key**  
Key: 7VZ2LM81Z79B8KOQ  
Generate New Write API Key

**Read API Keys**  
Key: 3K4MYLKUQPYQ1WYF  
Note:  
Save Note Delete API Key

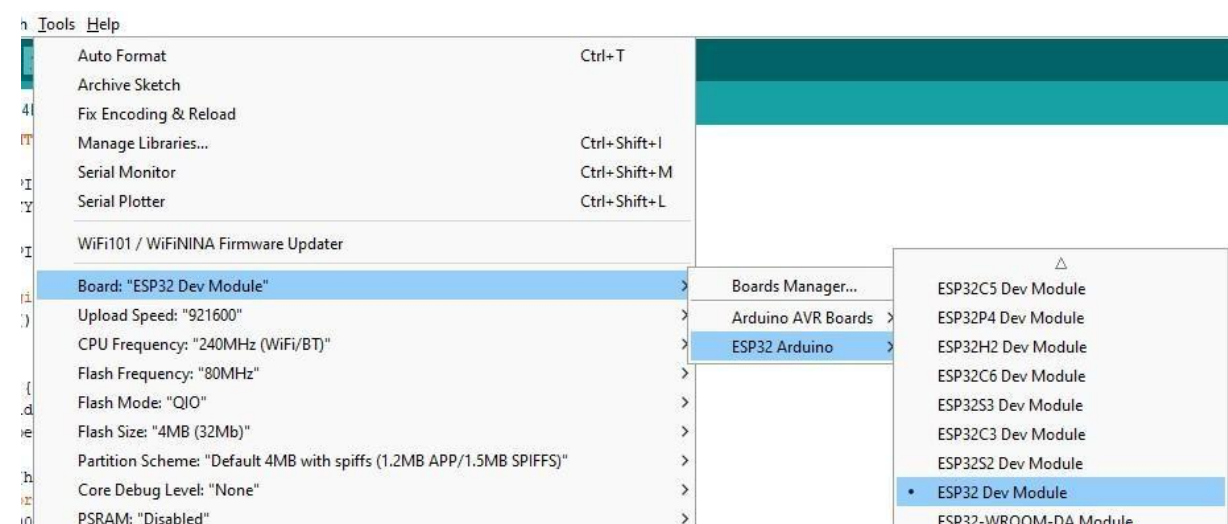
**Help**  
API keys en generated v  
API Ke:  
• Write: compi  
• Read: Click C  
• Note: keep t  
API Re:  
Write a ( GET htt  
Read a C

Open arduino app:

Sketch - include library- manage libraries- search: DHT Sensor Library



And Install.



Code:

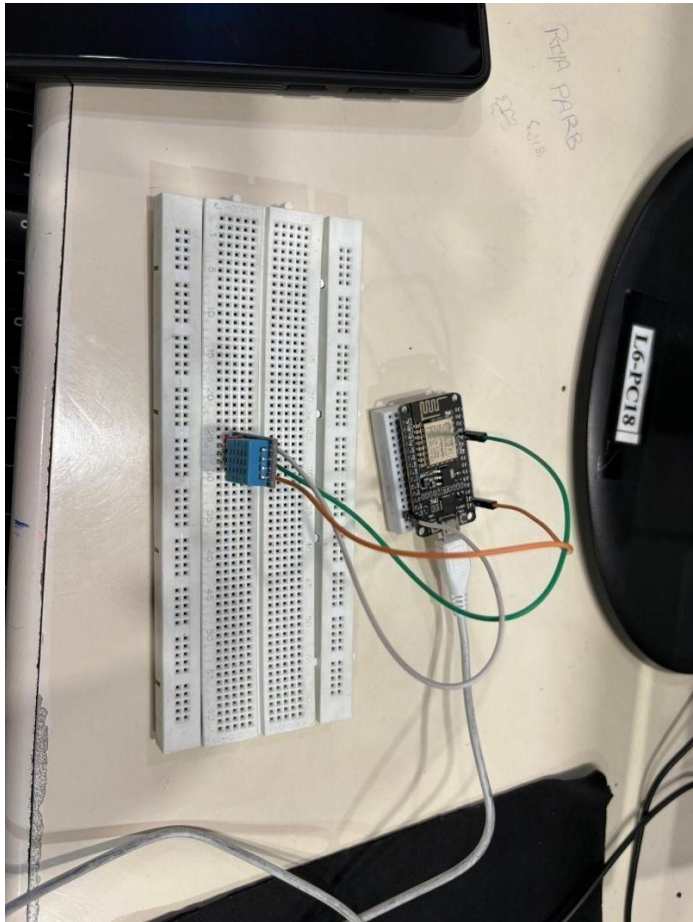
sketch\_sep3a.ino

```
1  #include <DHT.h>
2  #define DHTPIN 4
3  #define DHTTYPE DHT11
4  DHT dht(DHTPIN, DHTTYPE);
5
6  void setup() {
7    Serial.begin(115200);
8    dht.begin();
9  }
10 void loop() {
11   float humidity = dht.readHumidity();
12   float temperature = dht.readTemperature();
13
14   if (isnan(humidity) || isnan(temperature)) {
15     Serial.println("Failed to read from DHT11");
16     delay(2000);
17     return;
18   }
19   Serial.print("Temperature:");
20   Serial.print(temperature);
21   Serial.print("°C Humidity:");
22   Serial.print(humidity);
23   Serial.println("%");
24
25   delay(2000);
26 }
```

Output:

COM3

```
11:05:47.388 -> Temperature: 31.10°C Humidity: 76.30%
11:05:49.394 -> Temperature: 31.10°C Humidity: 76.20%
11:05:51.439 -> Temperature: 31.10°C Humidity: 76.20%
11:05:53.443 -> Temperature: 31.10°C Humidity: 76.20%
11:05:55.495 -> Temperature: 31.10°C Humidity: 76.20%
11:05:57.491 -> Temperature: 31.20°C Humidity: 76.20%
11:05:59.554 -> Temperature: 31.20°C Humidity: 76.20%
11:06:01.547 -> Temperature: 31.10°C Humidity: 76.20%
11:06:03.592 -> Temperature: 31.20°C Humidity: 76.10%
11:06:05.591 -> Temperature: 31.20°C Humidity: 76.00%
11:06:07.637 -> Temperature: 31.20°C Humidity: 76.00%
11:06:09.650 -> Temperature: 31.20°C Humidity: 75.90%
11:06:11.687 -> Temperature: 31.20°C Humidity: 75.90%
11:06:13.734 -> Temperature: 31.10°C Humidity: 75.90%
11:06:15.714 -> Temperature: 31.20°C Humidity: 75.90%
```



## Practical No 3

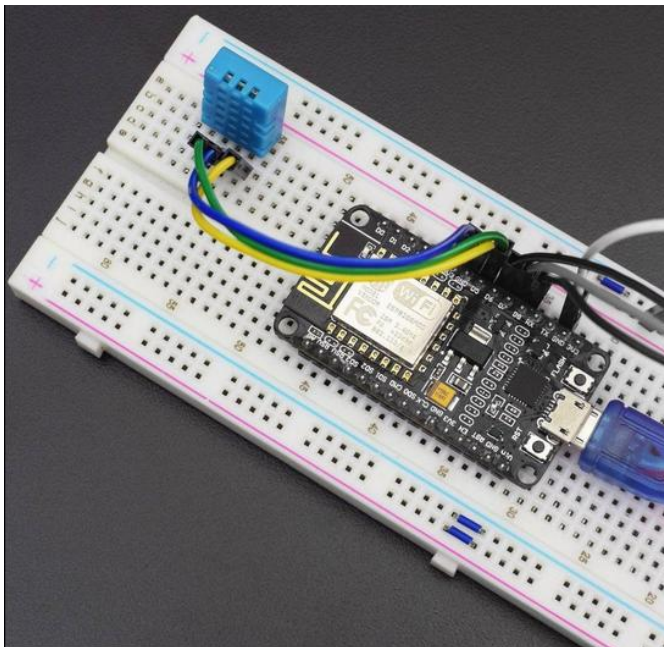
**Aim: To write real-time temperature and humidity data to the ThingSpeak cloud using ESP8266 and a Write API Key.**

### Theory

ThingSpeak is an IoT cloud platform used to collect, store, visualize and analyze sensor data. ESP8266 connects to Wi-Fi and uploads DHT11 readings to a ThingSpeak channel. The channel can contain separate fields for temperature and humidity. ThingSpeak dashboards can display the collected values as graphs and gauges.

### Hardware Components

8. ESP8266 NodeMCU
9. DHT11 sensor
10. Breadboard
11. Jumper wires
12. USB cable
13. Wi-Fi connection



### Code

```
#include <ESP8266WiFi.h>
#include <ThingSpeak.h>
#include <DHT.h>
const char* ssid = "YOUR_WIFI";
const char* password = "YOUR_PASSWORD";
unsigned long channelID = YOUR_CHANNEL_ID;
const char* writeAPIKey = "YOUR_WRITE_API_KEY";
#define DHTPIN D4
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);
WiFiClient client;
void setup() {
  Serial.begin(9600);
  dht.begin();
  WiFi.begin(ssid, password);
  Serial.print("Connecting to WiFi");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println();
```

```
Serial.println("WiFi Connected");
ThingSpeak.begin(client);
void loop() {
  float temperature = dht.readTemperature();
  float humidity = dht.readHumidity();
  ThingSpeak.setField(1, temperature);
  ThingSpeak.setField(2, humidity);
  int response = ThingSpeak.writeFields(
    channelID,
    writeAPIKey );
  Serial.print("Temperature: ");
  Serial.println(temperature);
  Serial.print("Humidity: ");
  Serial.println(humidity);
  if (response == 200) {
    Serial.println("Data uploaded successfully");}
  else {
    Serial.print("Upload failed: ");
    Serial.println(response);
  }
  Serial.println("-----");
  delay(20000);
}
```

## Output



Figure: Serial Monitor Output

(Temperature and Humidity readings with successful upload)

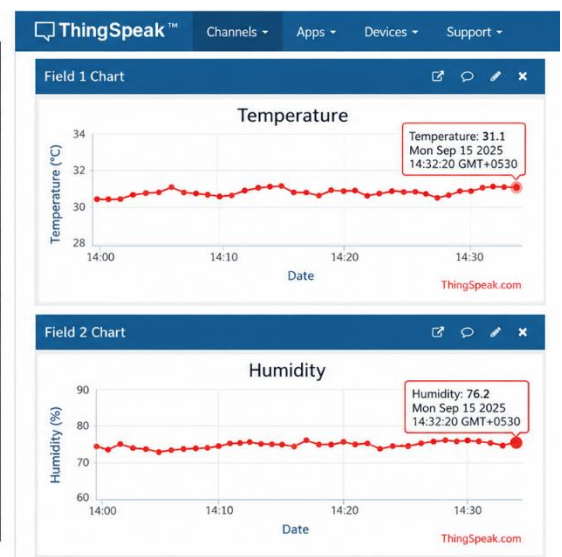


Figure: ThingSpeak Output

(Temperature and Humidity graphs)

## Observation

1. The ESP8266 successfully connected to the Wi-Fi network.
2. Real-time temperature and humidity readings from the DHT11 sensor are displayed on the Serial Monitor.
3. The data is uploaded successfully to the ThingSpeak cloud channel, and the graphs for temperature and humidity are generated in real-time.

## Conclusion

Thus, real-time DHT11 data was successfully uploaded to ThingSpeak using ESP8266.

## Practical 4

**Aim:** To retrieve the latest temperature and humidity data from ThingSpeak using the Read APIKey

**Sol:**

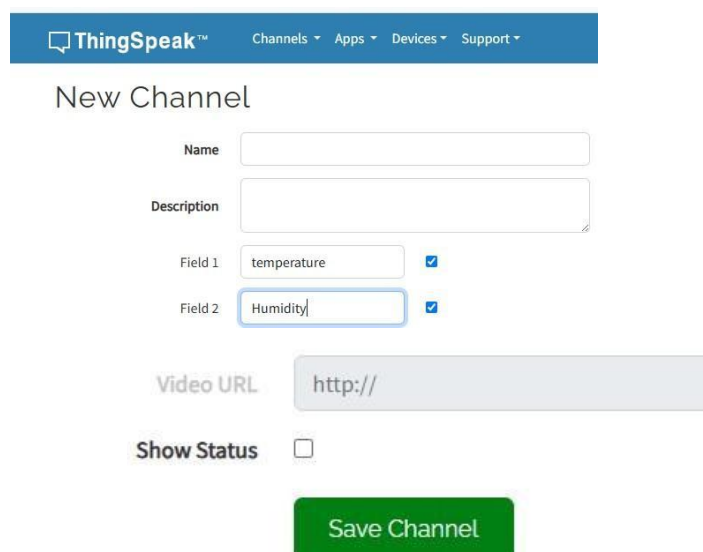
Open: <https://thingspeak.mathworks.com/>

Login with MATLAB Online.

After Login:

My channel:

New channel:



The screenshot shows the 'New Channel' form on the ThingSpeak website. The form includes fields for 'Name', 'Description', 'Field 1' (set to 'temperature'), 'Field 2' (set to 'Humidity'), and 'Video URL' (set to 'http://'). There are checkboxes for 'Show Status' and 'Save Channel'.

Save channel.

Go to APIKeys:

Read a Channel Feed

```
GET https://api.thingspeak.com/channels/3486981/feeds.json?api_key=3K4MYLKUQPYQ1WY
```

**Code:**

```
#include <WiFi.h>
```

```
#include <HTTPClient.h>
```

```
#include <DHT.h>
```

```
#define DHTPIN 4
```

```
#define DHTTYPE DHT11

#define LED_PIN 23

DHT dht(DHTPIN, DHTTYPE);

const char* ssid = "Pranali"; const
char* password = "1234567890";

String readAPIKey = "3K4MYLKUQPYPYQ1WYF";
const char* server =
"https://api.thingspeak.com/update";

const float TEMP_THRESHOLD = 30.0;

void setup() {
  Serial.begin(115200); dht.begin();

  pinMode(LED_PIN, OUTPUT);
  digitalWrite(LED_PIN, LOW);

  WiFi.begin(ssid, password);
  Serial.print("Connecting to Wi-Fi");

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println();
  Serial.println("Wi-Fi Connected!");
}
```

```
void loop() { if (WiFi.status() ==  
WL_CONNECTED) {  
  
    float humidity = dht.readHumidity();  
    float temperature = dht.readTemperature();  
  
    if (isnan(humidity) || isnan(temperature)) {  
        Serial.println("Failed to read DHT11");  
        delay(2000);    return;  
    }  
  
    Serial.print("Temperature: ");  
    Serial.print(temperature);  
    Serial.print(" °C, Humidity: ");  
    Serial.print(humidity);  
    Serial.println(" %");  
  
    // Temperature-based LED control    if  
(temperature >= TEMP_THRESHOLD) {  
        digitalWrite(LED_PIN, HIGH);  
        Serial.println("High temperature - LED ON");  
    } else {  
        digitalWrite(LED_PIN, LOW);  
        Serial.println("Normal temperature - LED OFF");  
    }  
  
    // Upload the readings to ThingSpeak  
    HTTPClient http;  
  
    String url = String(server) +  
        "?api_key=" + writeAPIKey +
```

```
"&field1=" + String(temperature) +  
"&field2=" + String(humidity);
```

```
http.begin(url); int  
httpCode = http.GET();
```

```
Serial.print("ThingSpeak response: ");  
Serial.println(httpCode);
```

```
http.end();  
}
```

```
delay(20000);  
}
```

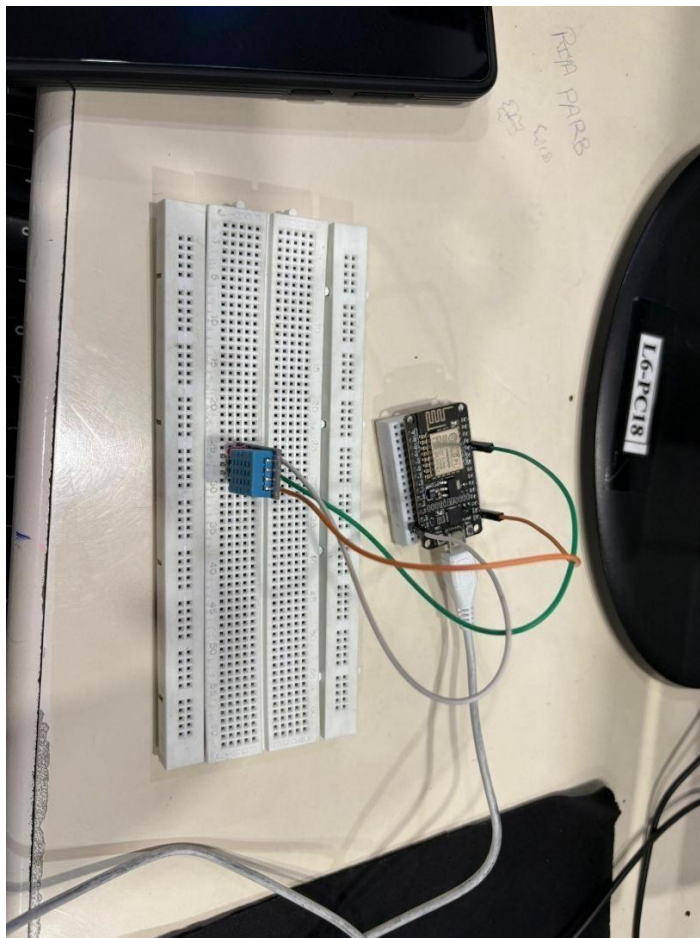
## Output:

```
api.thingspeak.com/channels/3486981/feeds.json?api_key=3K4MYLKUQPYPYQ1WYF&results=2

Continue where you left off: Chrome can restore your tabs eve

Pretty-print ☒

{
  "channel": {
    "id": 3486981,
    "name": "Channel 3486981",
    "latitude": "0.0",
    "longitude": "0.0",
    "field1": "temperature",
    "field2": "humidity",
    "created_at": "2026-09-09T05:42:27Z",
    "updated_at": "2026-09-09T05:42:27Z",
    "last_entry_id": 17
  },
  "feeds": [
    {
      "created_at": "2026-09-09T05:54:29Z",
      "entry_id": 16,
      "field1": "31.30",
      "field2": "77.10"
    },
    {
      "created_at": "2026-09-09T05:54:51Z",
      "entry_id": 17,
      "field1": "31.30",
      "field2": "77.10"
    }
  ]
}
```



## Practical 5

**AIM: To control a lamp indicator on ThingSpeak using ESP8266 through cloud-based control.**

```
#include <WiFi.h>

#include <WiFiClient.h>

const char* ssid = "realme 8 5G";
const char* pass = "12345678";

// Choose an ADC-capable pin for LDR input (e.g. GPIO34)
#define LDR_PIN 34

char host[] = "api.thingspeak.com";

String ApiKey = "IS32ZE3XYQ9LX80P"; // Your ThingSpeak Write API key
String path = "/update?key=" + ApiKey + "&field1=";

int ldr_value = 0;

void setup() {
  Serial.begin(115200)
  ; Serial.println();

  Serial.print("Connecting to ");
  Serial.println(ssid);
  WiFi.begin(ssid, pass);

  // Wait for WiFi connection
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");

    }

  Serial.println();
  Serial.println("WiFi connected!");
  Serial.print("IP address: ");
  Serial.println(WiFi.localIP());
```

```
pinMode(LDR_PIN, INPUT);
}

void loop() {
  ldr_value = analogRead(LDR_PIN); // Read LDR value (0–4095)
  Serial.print("LIGHT INTENSITY VALUE = ");
  Serial.println(ldr_value);

  WiFiClient client;
  const int httpPort = 80;

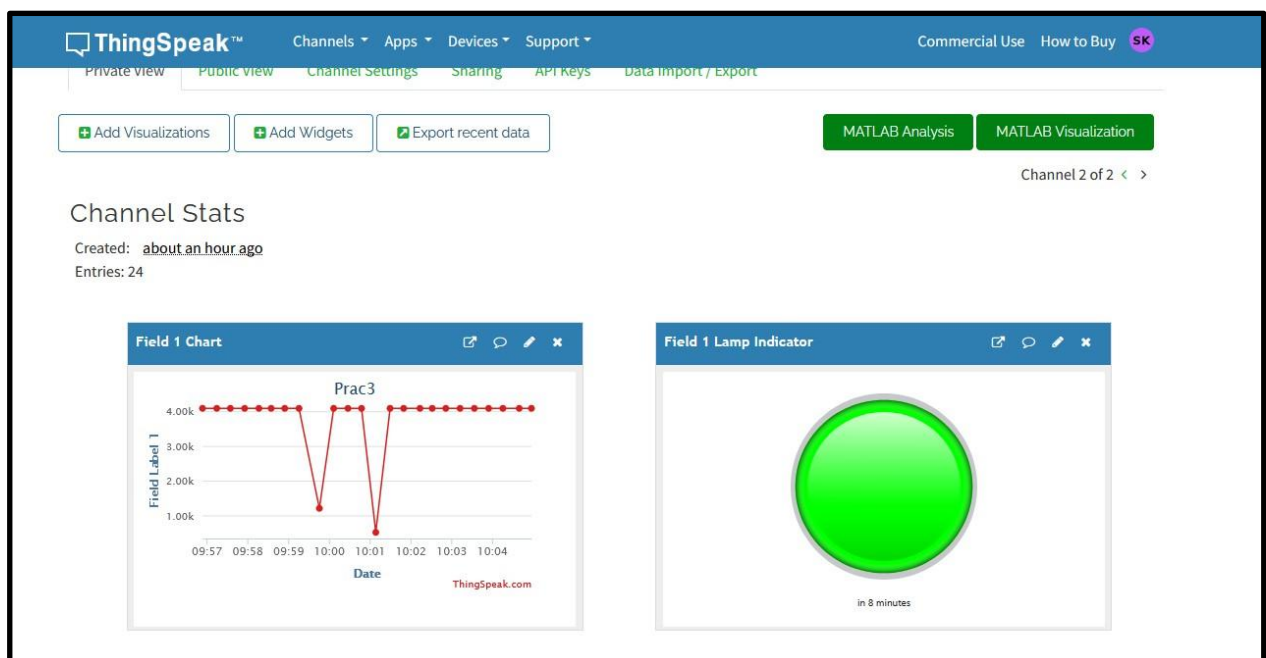
  if (!client.connect(host, httpPort)) {
    Serial.println("Connection to ThingSpeak failed");
    return;
  }

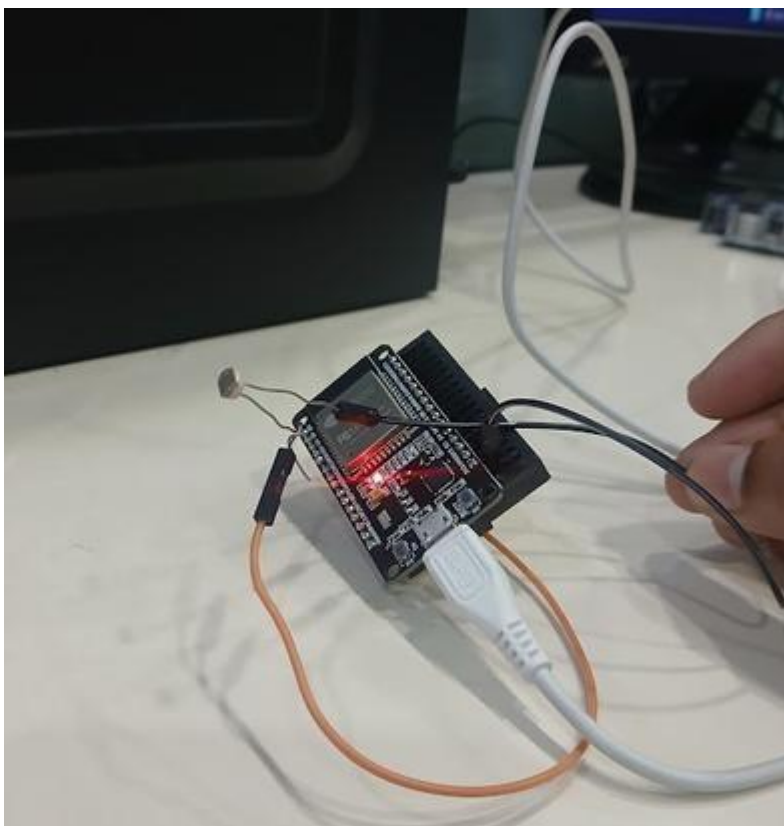
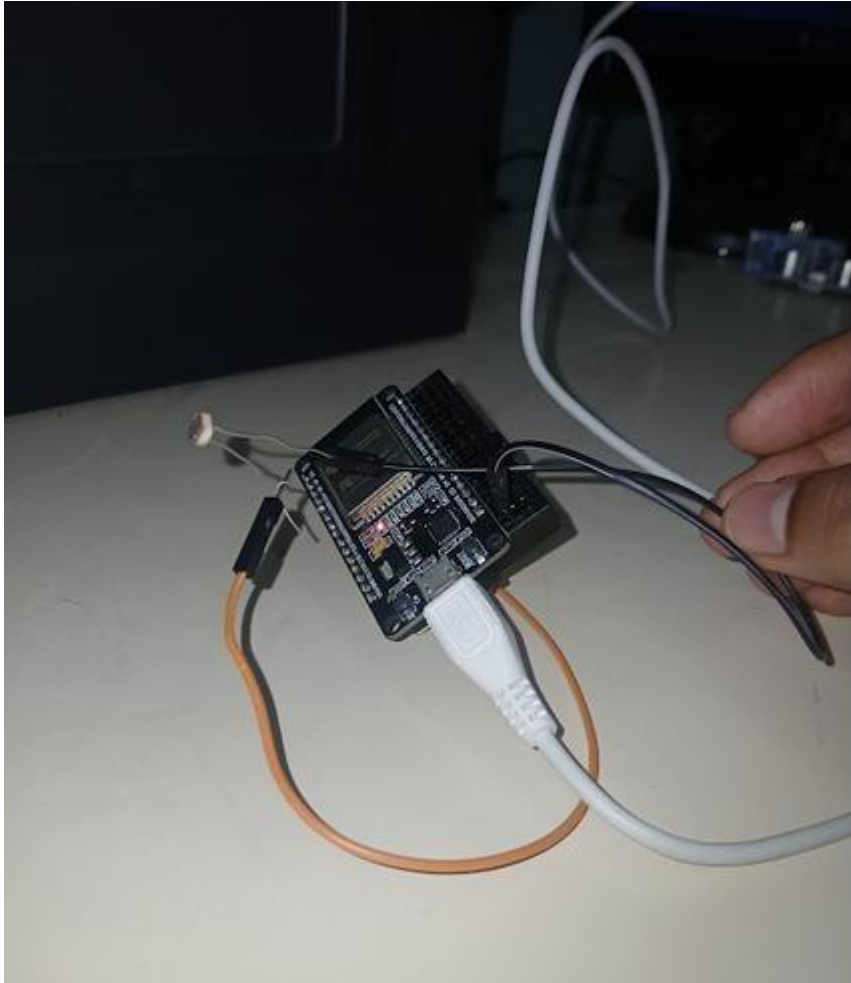
  // Send GET request
  client.print(String("GET ") + path + ldr_value + " HTTP/1.1\r\n" +
    "Host: " + host + "\r\n" +
    "Connection: keep-alive\r\n\r\n");

  delay(20000); // ThingSpeak accepts updates every 15+ seconds
}
```

```
COM3
IP address: 10.144.75.82
LIGHT INTENSITY VALUE = 4095
LIGHT INTENSITY VALUE = 4095
LIGHT INTENSITY VALUE = 4095
LIGHT INTENSITY VALUE = 243
Connection to ThingSpeak failed
LIGHT INTENSITY VALUE = 1215
LIGHT INTENSITY VALUE = 4095
LIGHT INTENSITY VALUE = 4095
LIGHT INTENSITY VALUE = 4095
LIGHT INTENSITY VALUE = 528
LIGHT INTENSITY VALUE = 4095
LIGHT INTENSITY VALUE = 4095
```

☒ Autoscroll ☐ Show timestamp Newline 115200 baud Clear output





## Practical No 6

**Aim: To interface an LDR sensor with ESP32 and automatically control an LED based on light intensity.**

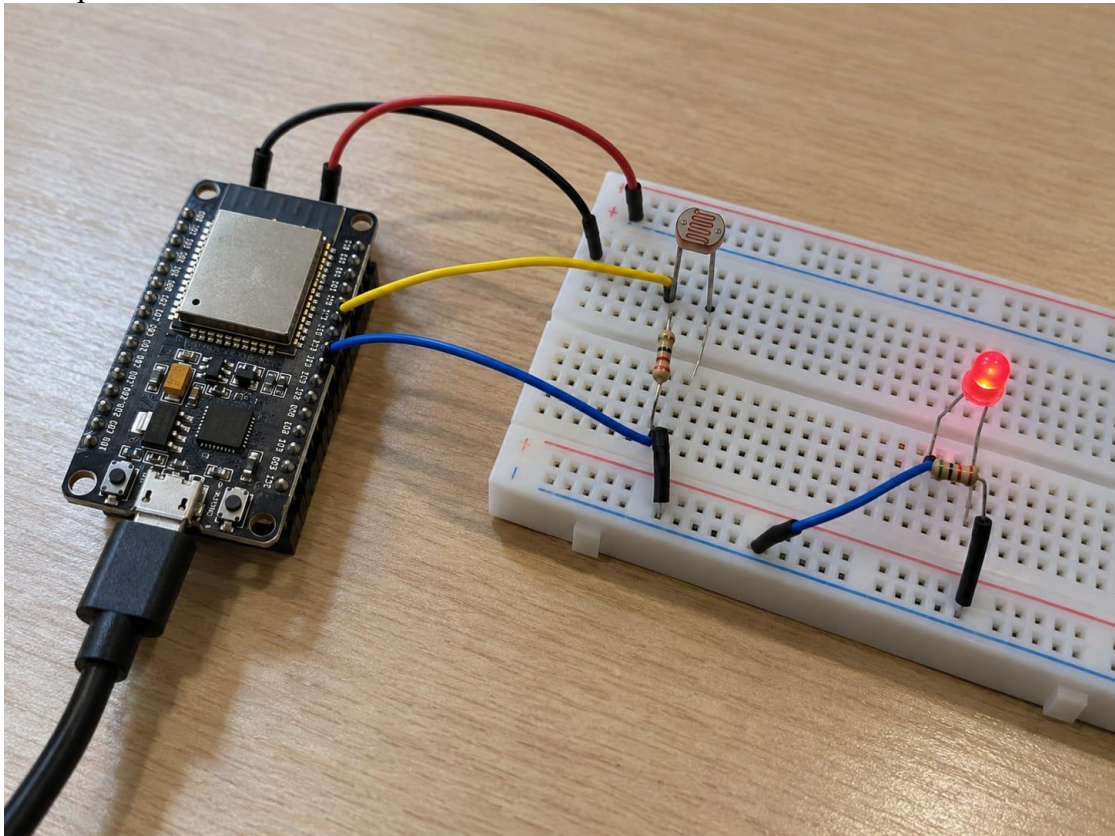
### Theory

An LDR or Light Dependent Resistor changes its resistance according to the intensity of light. The ESP32 reads the corresponding analog voltage using an ADC pin. When the light intensity crosses a predefined threshold, the ESP32 switches the LED ON or OFF.

ESP32 examples commonly use an ADC-capable GPIO to read the light sensor and a digital GPIO to control the LED.

### Hardware Components

1. ESP32 development board
2. LDR
3. 10k $\Omega$  resistor
4. LED
5. 220 $\Omega$  resistor
6. Breadboard
7. Jumper wires



**Figure: ESP32 LDR sensor and LED circuit**

### Code

```
int ldrPin = 34;
int ledPin = 2;
void setup() {
  Serial.begin(115200);
  pinMode(ledPin, OUTPUT);
}
void loop() {
  int lightValue = analogRead(ldrPin);
  Serial.print("Light Intensity: ");
  Serial.println(lightValue);
  if (lightValue < 1500) {
    digitalWrite(ledPin, HIGH);
    Serial.println("Dark - LED ON"); } }
```

```
else {digitalWrite(ledPin, LOW);  
Serial.println("Bright - LED OFF"); }  
delay(1000);}
```

## Output

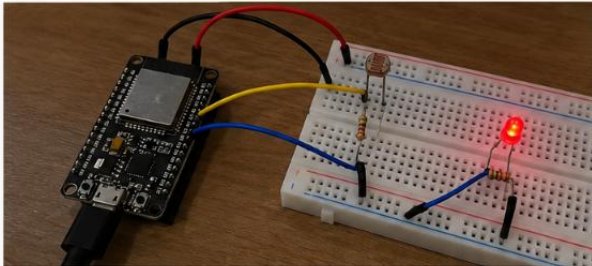


Figure 1: LED ON in dark condition (Low light intensity)

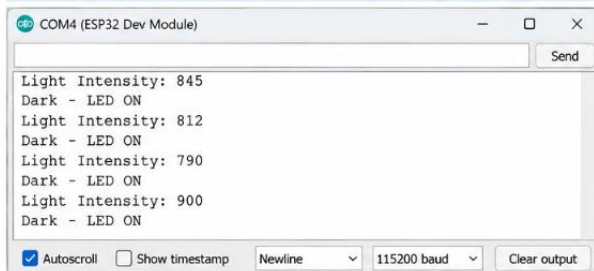


Figure 2: Serial Monitor Output (Dark condition)

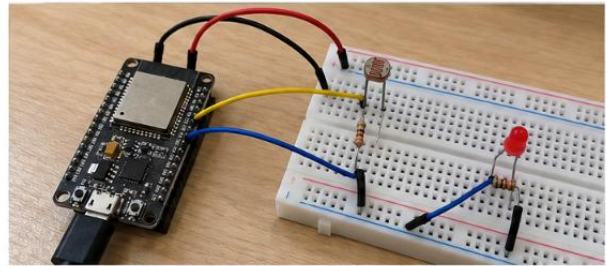


Figure 3: LED OFF in bright condition (High light intensity)

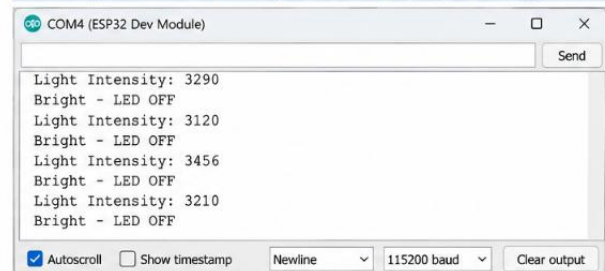


Figure 4: Serial Monitor Output (Bright condition)

1. When the light intensity is low (dark environment), the LDR resistance increases, the ESP32 reads a lower analog value and the LED turns ON.
2. When the light intensity is high (bright environment), the LDR resistance decreases, the ESP32 reads a higher analog value and the LED turns OFF.
3. The Serial Monitor displays the real-time light intensity values along with the LED status.

## Conclusion

Thus, the ESP32 successfully controlled the LED automatically according to light intensity

## Practical 7

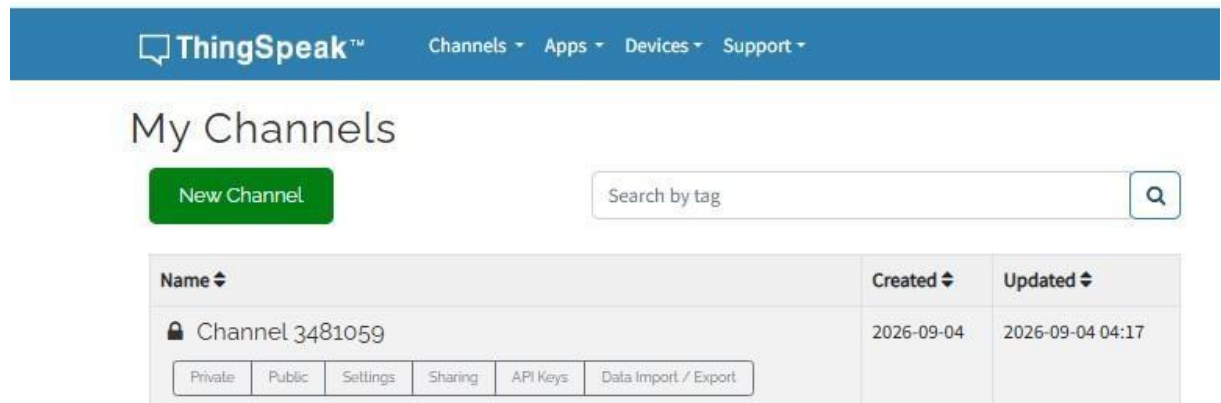
**Aim:** To control an LED using ESP32 based on a predefined temperature threshold.

**Sol:**

**Open:** <https://thingspeak.mathworks.com/>

**Login with MATLAB Online.**

**After Login:**

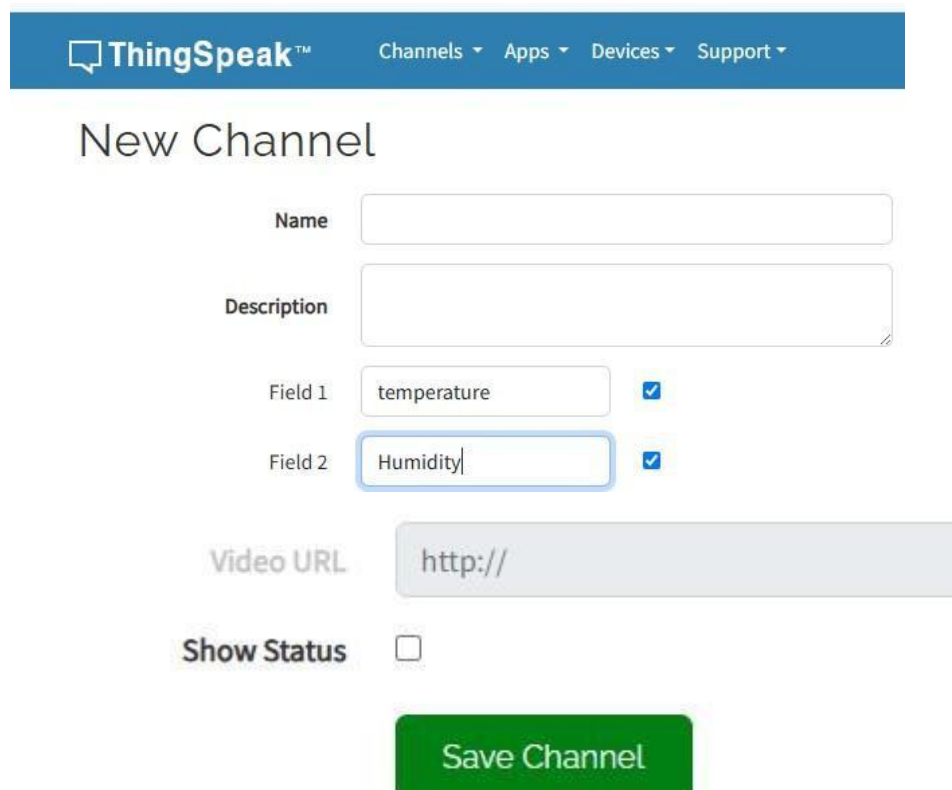


ThingSpeak™ Channels ▾ Apps ▾ Devices ▾ Support ▾

### My Channels

[New Channel](#)

| Name ▾                                                                                                                                                                           | Created ▾  | Updated ▾        |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|------------------|
| Channel 3481059<br><a href="#">Private</a> <a href="#">Public</a> <a href="#">Settings</a> <a href="#">Sharing</a> <a href="#">API Keys</a> <a href="#">Data Import / Export</a> | 2026-09-04 | 2026-09-04 04:17 |



ThingSpeak™ Channels ▾ Apps ▾ Devices ▾ Support ▾

### New Channel

Name

Description

Field 1  ☒

Field 2  ☒

Video URL

Show Status ☐

[Save Channel](#)

## Click on API:

ThingSpeak™

Channels ▾ Apps ▾ Devices ▾ Support ▾

Commercial Use How to Buy AS

Channel 3481059

Channel ID: 3481059  
Author: mwa0000038094008  
Access: Private

Private View Public View Channel Settings Sharing API Keys Data Import / Export

Add Visualizations Add Widgets Export recent data

MATLAB Analysis MATLAB Visualization

Channel Stats

Created: less than a minute ago

ThingSpeak™

Channels ▾ Apps ▾ Devices ▾ Support ▾

Author: mwa0000038124079  
Access: Private

Private View Public View Channel Settings Sharing API Keys Data Import / Export

Write API Key

Key 7VZ2LM81Z79B8KOQ

Generate New Write API Key

Read API Keys

Key 3K4MYLKUQPYQ1WYF

Note

Save Note Delete API Key

Help

API keys en  
generated v

API Ke

- Write ,  
compi
- Read /  
Click C
- Note:  
keep t

API Re

Write a t

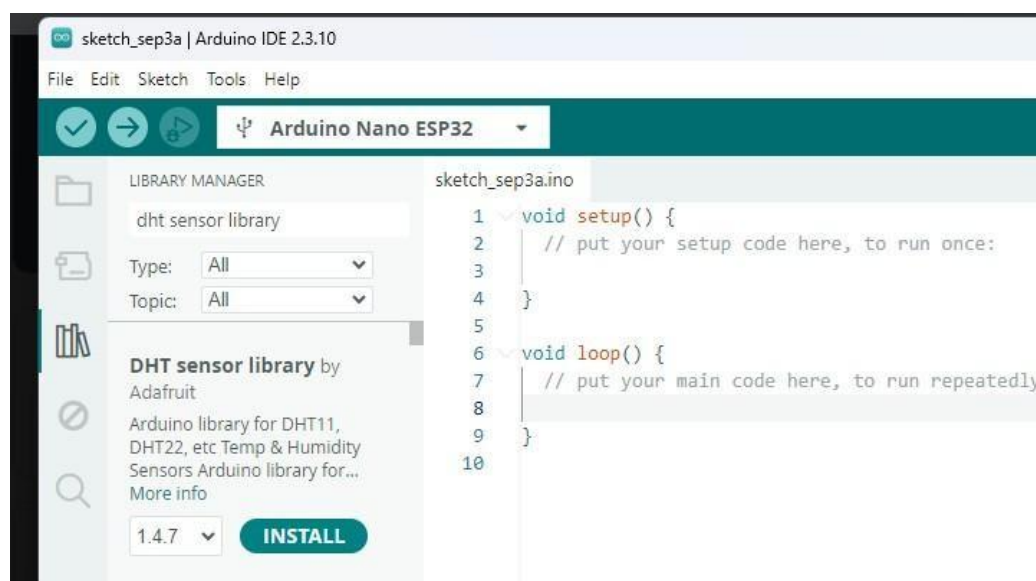
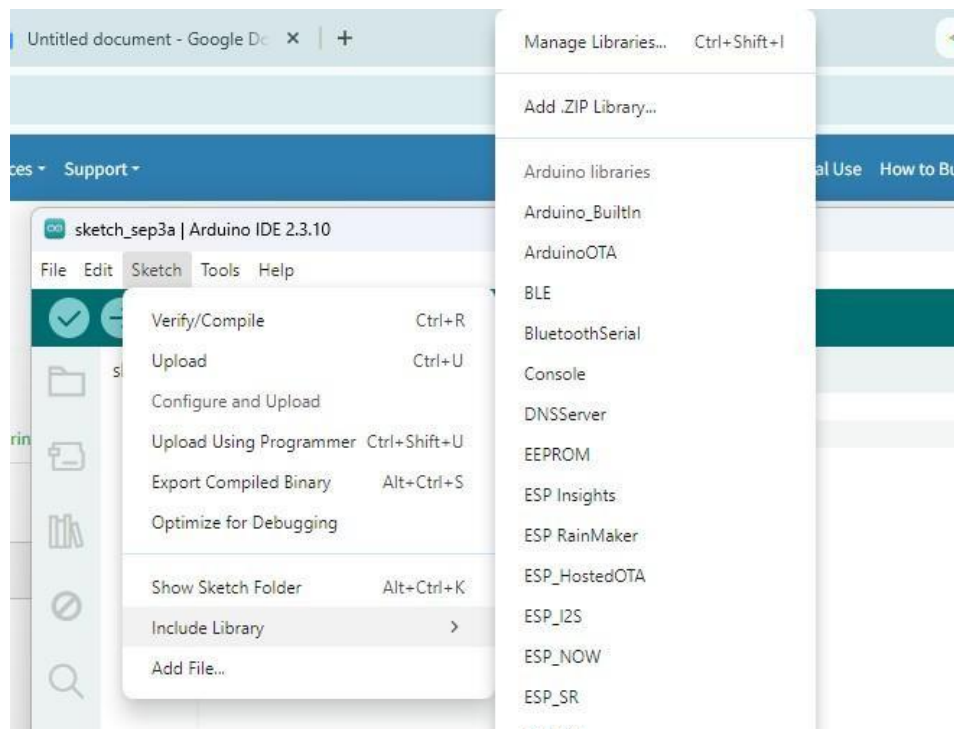
GET htt

Read a C

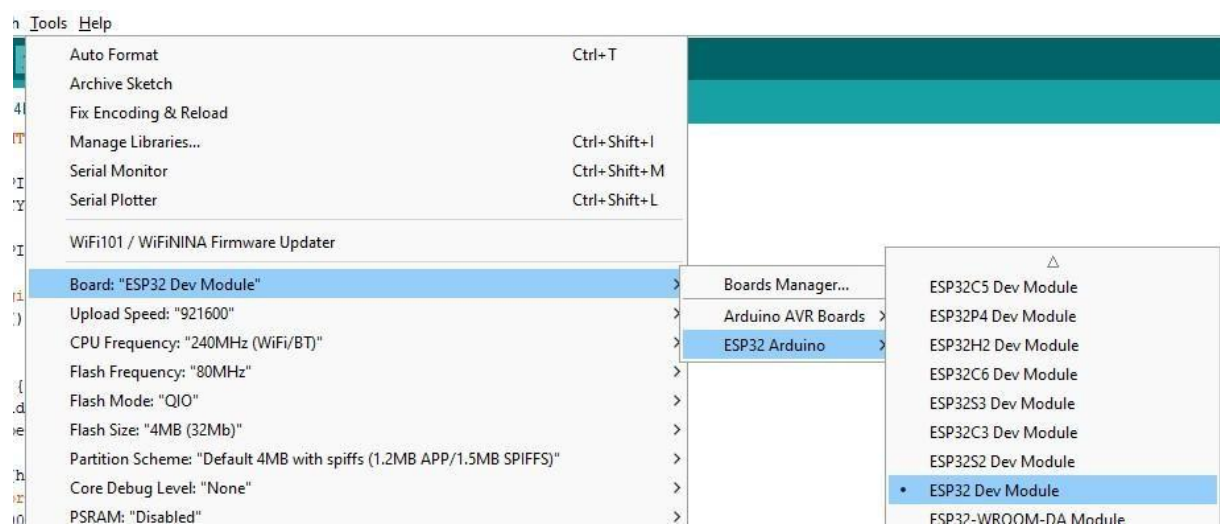
GET htt

**Open arduino app:**

**Sketch - include library- manage libraries- search: DHT Sensor Library**



**And Install.**



**Code:**

```
dht.begin();

pinMode(LED_PIN, OUTPUT);
digitalWrite(LED_PIN, LOW);

WiFi.begin(ssid, password);
Serial.print("Connecting to Wi-Fi");

while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
}

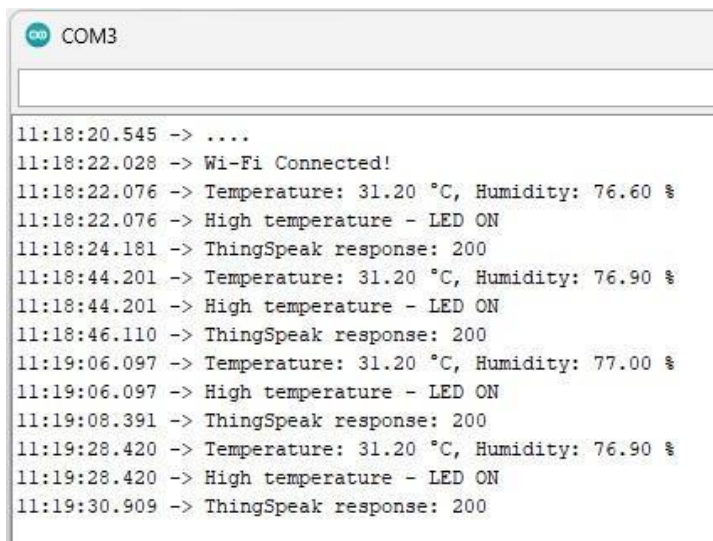
Serial.println();
Serial.println("Wi-Fi Connected!");
}

void loop() { if (WiFi.status() ==
WL_CONNECTED) { float humidity =
```

```
dht.readHumidity();    float temperature =  
dht.readTemperature();  
  
    if (isnan(humidity) || isnan(temperature)) {  
Serial.println("Failed to read DHT11");  
delay(2000);    return;  
    }  
  
    Serial.print("Temperature: ");  
    Serial.print(temperature);  
    Serial.print(" °C, Humidity: ");  
    Serial.print(humidity);  
    Serial.println(" %");  
  
    // Temperature-based LED control    if  
(temperature >= TEMP_THRESHOLD) {  
digitalWrite(LED_PIN, HIGH);  
    Serial.println("High temperature - LED ON");  
    } else {  
    digitalWrite(LED_PIN, LOW);  
    Serial.println("Normal temperature - LED OFF");  
    }  
  
    // Upload the readings to ThingSpeak  
    HTTPClient http;  
  
    String url = String(server) +  
        "?api_key=" + writeAPIKey +  
        "&field1=" + String(temperature) +  
        "&field2=" + String(humidity);
```

```
    http.begin(url);    int  
    httpCode = http.GET();  
  
    Serial.print("ThingSpeak response: ");  
    Serial.println(httpCode);  
  
    http.end();  
}  
  
delay(20000);  
}
```

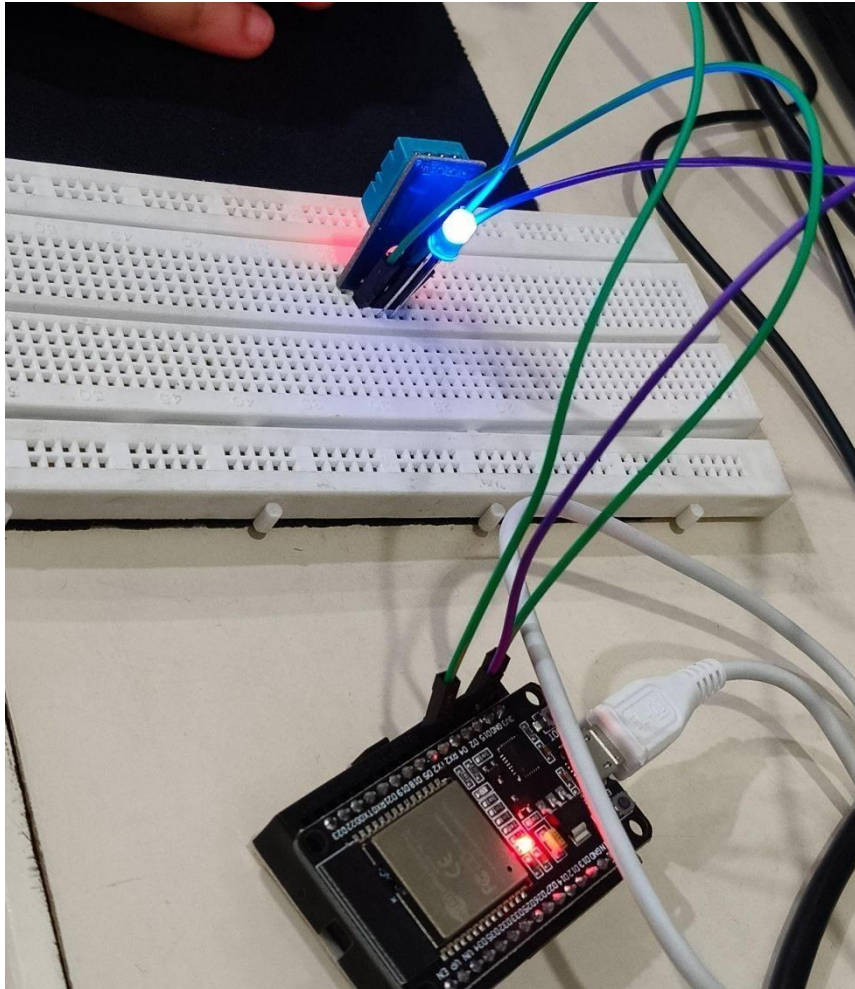
### Output:



```
COM3  
11:18:20.545 -> ....  
11:18:22.028 -> Wi-Fi Connected!  
11:18:22.076 -> Temperature: 31.20 °C, Humidity: 76.60 %  
11:18:22.076 -> High temperature - LED ON  
11:18:24.181 -> ThingSpeak response: 200  
11:18:44.201 -> Temperature: 31.20 °C, Humidity: 76.90 %  
11:18:44.201 -> High temperature - LED ON  
11:18:46.110 -> ThingSpeak response: 200  
11:19:06.097 -> Temperature: 31.20 °C, Humidity: 77.00 %  
11:19:06.097 -> High temperature - LED ON  
11:19:08.391 -> ThingSpeak response: 200  
11:19:28.420 -> Temperature: 31.20 °C, Humidity: 76.90 %  
11:19:28.420 -> High temperature - LED ON  
11:19:30.909 -> ThingSpeak response: 200
```

**Big - Data**

**Small- gnd**

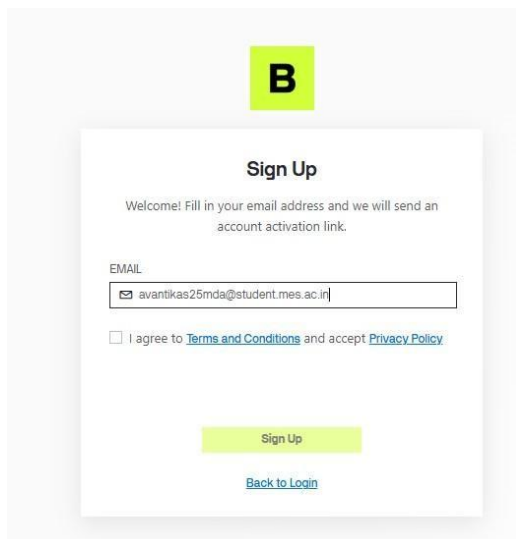
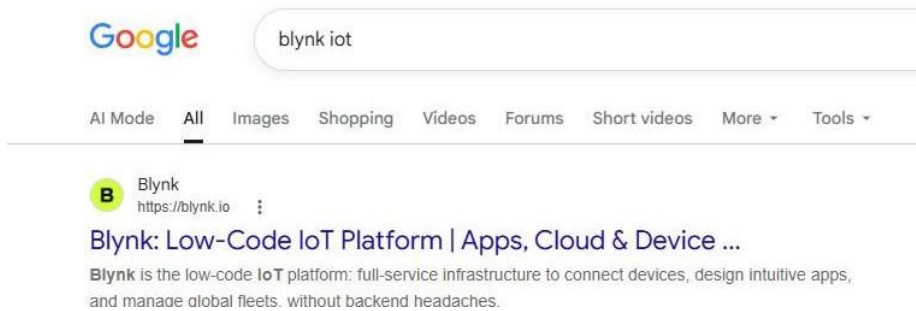


## Practical 8

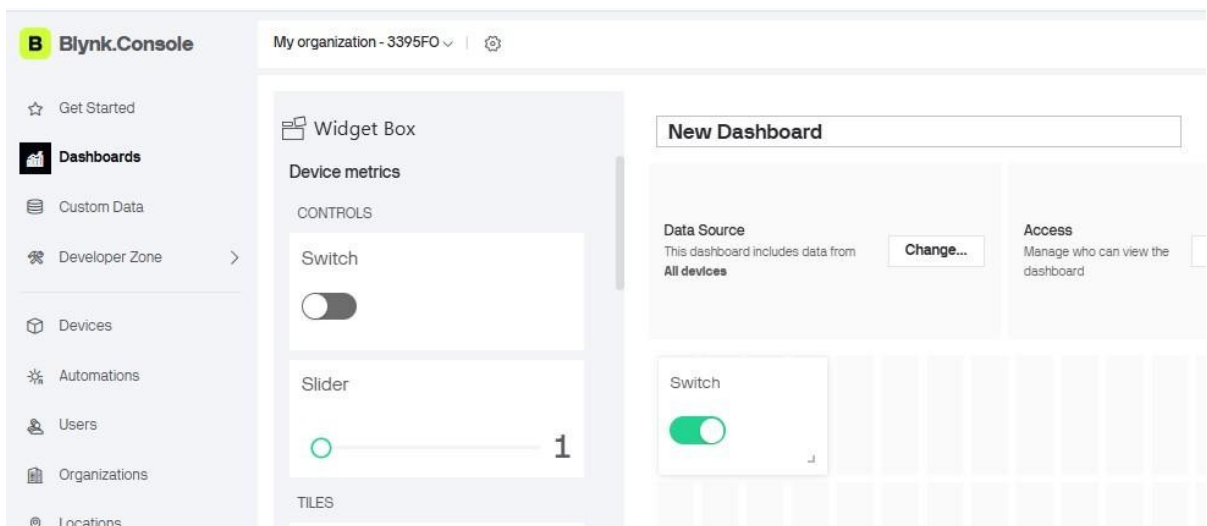
**Aim: To interact with web services and control devices using**

**(Blynk.) Sol:**

**Search: Blynk iot**



**Create dashboard:**



## Create template:

**B** Blynk.Console

My organization - 3395FO

Messages used 0 of 100.0k

Ask AI

Get Started

Dashboards

Custom Data

**Developer Zone**

Devices

Automations

Users

Organizations

Locations

Snapshots

Fleet Management

In-App Messaging

Predictive Maintenance

DEVELOPER ZONE

**My Templates**

Blueprints

Assets

Blynk Air (OTA)

Static Tokens

Rule Engine

OAuth 2.0

Webhooks

Integrations

Debugging

Provisioning Sessions

Crash Reports

Errors

Start by creating your first template

Template is a digital model of a physical object. It is used in Blynk platform as a template to be assigned to devices.

+ New Template

### Create New Template

NAME

led control

11 / 50

HARDWARE

ESP32

CONNECTION TYPE

WIFI

DESCRIPTION

Description

0 / 128

Cancel

Done



### New Datastream

**General**  
Semantic  
Units  
History  
Expiration  
Sync  
Automations  
Visibility  
Notes  
API reference

#### General

DATASTREAM NAME  
LED → DISPLAY NAME  
LED  
For your reference and for MQTT topic naming Shown in widgets, dashboards and reports.

VIRTUAL PIN  
V0 COLOR  
 ICON  


DATA TYPE  
Integer Double String Enum Location

MIN VALUE  
0 MAX VALUE  
1 DEFAULT VALUE  
0

#### Semantic

Semantic tags enable Blynk to understand the meaning of the datastream. This will be used in data aggregations, dashboards, automations, and beyond.

LED × + Custom tag

Cancel Save

My organization - 3395FO Messages used: 0 of 100.0k Ask AI

LED CONTROL

Home Datastreams Dashboard Events & Notifications Connection Lifecycle Data Converters

led control  
# Template ID: TMPL31FMxvC85

Cancel Save

Datastreams

Search datastream...

Browse Presets New Datastream

Alt 1 Integer Double String Enum Location Digital Analog

|  | Pin | Name | Data type | Semantics | Min | Max | Default | Units | Au | Actions |
|--|-----|------|-----------|-----------|-----|-----|---------|-------|----|---------|
|  | V0  | LED  | Integer   | LED       | 0   | 1   | 0       | -     |    | ...     |

## Devices:

**B Blynk.Console**  
Get Started  
Dashboards  
Custom Data  
Developer Zone  
**Devices**  
Automations  
Users  
Organizations  
Locations  
Snapshots

My organization - 3395FO

Create your first device  
We'll help you get your first device online  
+ New Device

### Select from template:

### New Device

Choose a way to create new device

Create New

From template

Manual entry

 Point on the cards to see instructions

Cancel

### New Device

Create new device by filling in the form below

TEMPLATE

led control

DEVICE NAME

led control 11 / 50

Cancel Create

```
#define BLYNK_TEMPLATE_NAME "led control"
```

```
#define BLYNK_AUTH_TOKEN "7CCE6CYI4cV7CF0q1sqlT5at-PVk9dpy"
```

My organization - 3395FO

Messages used: 0 of 100.0k

Ask AI

+ Add Tag

led control

Auth Token: 7CCE6CYI-\*\*\*\*

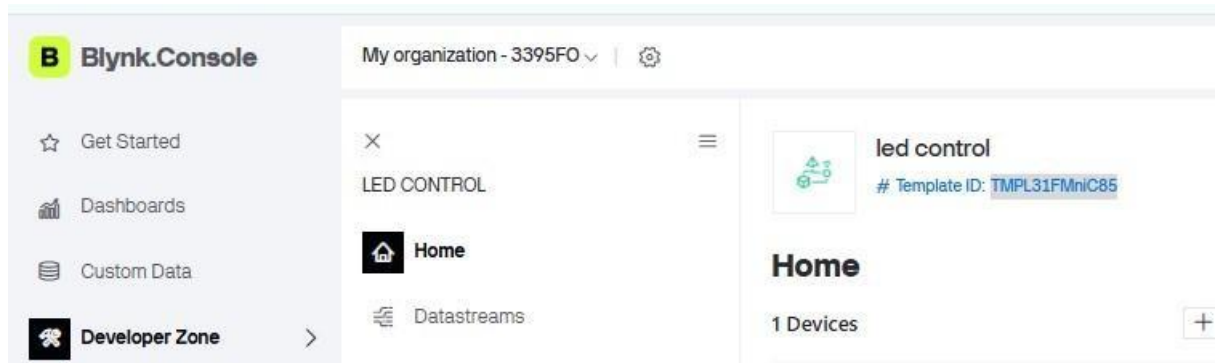
Avantika

My organization - 3395FO

Edit

Live 1h 6h 1d 1w 1mo 3mo 6mo 1y

TemplateID:

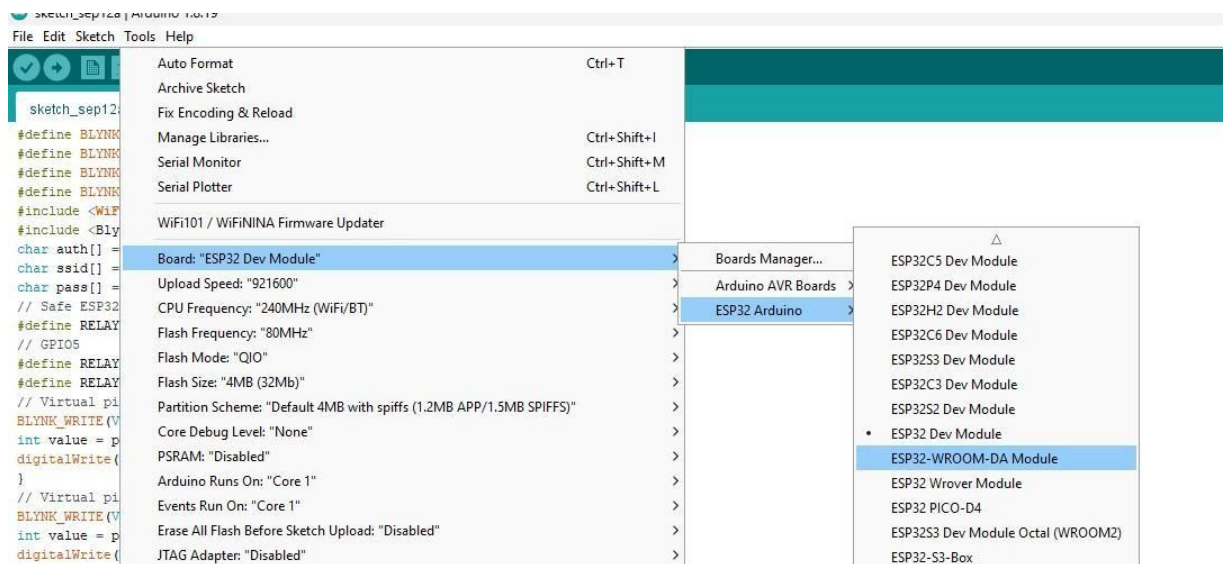


Open arduino:

Install library: blynk



Select tools:



### Connection:

**Led- esp**

**Small- gnd**

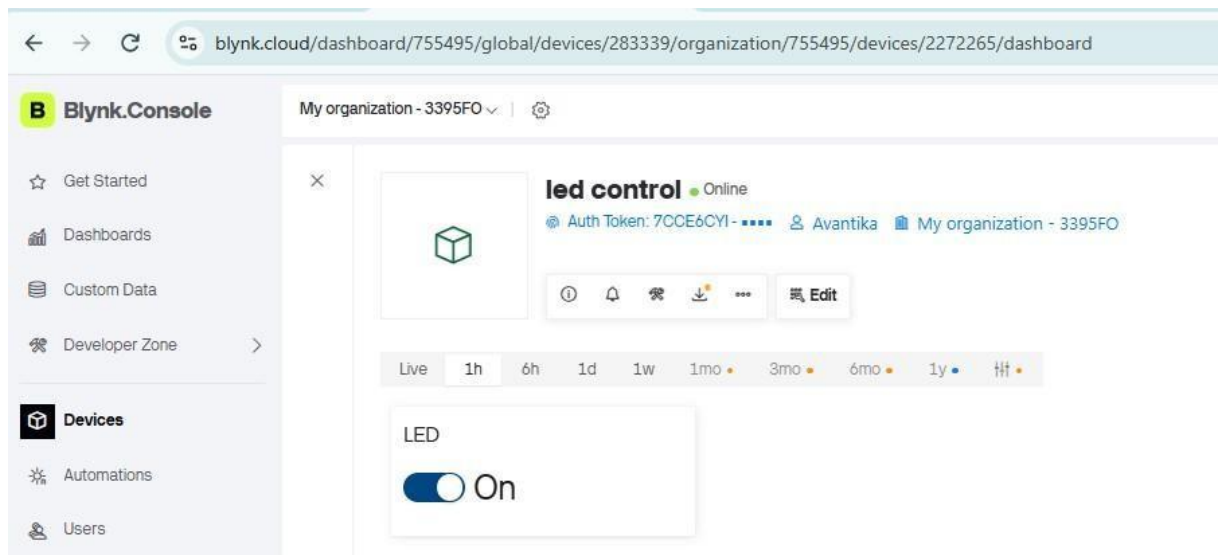
**Big - d4**

### Code

```
#define BLYNK_TEMPLATE_ID "TMPL31FMniC85"
#define BLYNK_TEMPLATE_NAME "led control"
#define BLYNK_AUTH_TOKEN "7CCE6CYI4cV7CF0q1sqlT5at-PVk9dpy"
#define BLYNK_PRINT Serial
#include <WiFi.h>
#include <BlynkSimpleEsp32.h> char
auth[] = BLYNK_AUTH_TOKEN;
char ssid[] = "Pranali"; char
pass[] = "1234567890";
// Safe ESP32 GPIO pins
#define RELAY1 4
// GPIO5
#define RELAY2 18 // GPIO18
#define RELAY3 19 // GPIO19
// Virtual pin V0 controls RELAY1
BLYNK_WRITE(V0){
int value = param.asInt(); digitalWrite(RELAY1,
value ? HIGH : LOW);
}
// Virtual pin V1 controls RELAY2
BLYNK_WRITE(V1){
int value = param.asInt(); digitalWrite(RELAY2,
value ? HIGH : LOW); }
// Virtual pin V2 controls RELAY3
```

```
BLYNK_WRITE(V2){  
  int value = param.asInt(); digitalWrite(RELAY3,  
  value ? HIGH : LOW);  
}  
void  
setup(){  
  pinMode(RELAY1, OUTPUT); pinMode(RELAY2,  
  OUTPUT); pinMode(RELAY3, OUTPUT);  
  // Initialize all relays to OFF  
  digitalWrite(RELAY1, LOW);  
  digitalWrite(RELAY2, LOW);  
  digitalWrite(RELAY3, LOW); Serial.begin(115200);  
  Blynk.begin(auth, ssid, pass);  
}  
void  
loop(){  
  Blynk.run();  
}
```

### Output:



## Practical 9

**Aim:** To calculate distance using Ultrasonic sensor

### Connection:

Ultrasonic Esp32

Vcc        3v3

Trig       D18

Echo       D19

Gnd        gnd

### Code:

```
int trigPin = 18;    int echoPin
= 19;    // TRIG pin // ECHO
pin float duration_us,
distance_cm; void setup() {
// begin serial port
Serial.begin (115200);
// configure the trigger pin to output mode pinMode(trigPin,
OUTPUT);
// configure the echo pin to input mode pinMode(echoPin,
INPUT);
} void loop()
{
// generate 10-microsecond pulse to TRIG pin
digitalWrite(trigPin, HIGH);
delayMicroseconds(10); digitalWrite(trigPin,
LOW);

// measure duration of pulse from ECHO pin duration_us
= pulseIn(echoPin, HIGH);
```

```
// calculate the distance distance_cm  
= 0.017 * duration_us;  
// print the value to Serial Monitor  
Serial.print("distance: ");  
Serial.print(distance_cm);  
Serial.println(" cm"); delay(500);  
}
```

### Output:

```
10:54:28.828 -> distance: 232.97 cm  
10:54:29.371 -> distance: 232.92 cm  
10:54:29.906 -> distance: 269.74 cm  
10:54:30.382 -> distance: 269.38 cm  
10:54:30.939 -> distance: 269.76 cm  
10:54:31.417 -> distance: 270.21 cm  
10:54:31.943 -> distance: 270.13 cm  
10:54:32.467 -> distance: 269.74 cm  
10:54:32.994 -> distance: 269.72 cm  
10:54:33.532 -> distance: 269.70 cm  
10:54:34.023 -> distance: 269.76 cm  
10:54:34.546 -> distance: 49.95 cm  
10:54:35.072 -> distance: 269.74 cm  
10:54:35.550 -> distance: 44.66 cm  
10:54:36.061 -> distance: 41.51 cm
```

## Observations

1. The ultrasonic sensor successfully measured and displayed the distance in centimeters on the Serial Monitor.
2. The distance initially increased from approximately 232.97 cm to 270.21 cm, indicating that the object was farther from the sensor.
3. The readings remained around 269–270 cm for several measurements.
4. The distance then decreased significantly to 49.95 cm, 44.66 cm, and 41.51 cm, indicating that the object moved closer to the sensor.
5. The Serial Monitor continuously displayed the changing distance values along with timestamps.

## Conclusion

Thus, the ESP32 successfully calculated and displayed the distance of an object using the ultrasonic sensor. The observed readings changed according to the movement and position of the object relative to the sensor.

## **Practical No 10**

### **Aim**

To monitor and sense the presence of an unauthorized person using a PIR sensor and ESP32, and to control an LED and display the motion status on the Serial Monitor.

### **Theory**

A PIR (Passive Infrared) sensor detects motion by sensing changes in infrared radiation produced by a moving person. The sensor provides a HIGH signal when motion is detected and a LOW signal when no motion is detected. The ESP32 reads the sensor output and controls an LED accordingly. The Serial Monitor displays the motion status for monitoring.

### **Hardware Components**

1. ESP32 development board
2. PIR motion sensor
3. LED
4. Breadboard
5. Jumper wires
6. USB cable

### **Code**

```
int led = 4; // the pin that the LED is attached to
int sensor = 15; // the pin that the sensor is attached to
int state = LOW; // by default, no motion detected
int val = 0; // variable to store the sensor status (value)

void setup() {
  pinMode(led, OUTPUT); // initialize LED as an output
  pinMode(sensor, INPUT); // initialize sensor as an input
  Serial.begin(115200); // initialize serial
  Serial.println("PIR Motion Sensor Test Started...");
}

void loop() {
  val = digitalRead(sensor); // read sensor value
  Serial.println(val); // debug: print raw sensor value

  if (val == HIGH) { // check if the sensor is HIGH
    digitalWrite(led, HIGH); // turn LED ON
    delay(100);

    if (state == LOW) {
      Serial.println("Motion detected!");
      state = HIGH;
    }
  }
  else {
    digitalWrite(led, LOW);
    delay(200);

    if (state == HIGH) {
      Serial.println("Motion stopped!");
      state = LOW;
    }
  }
}
```

## Output

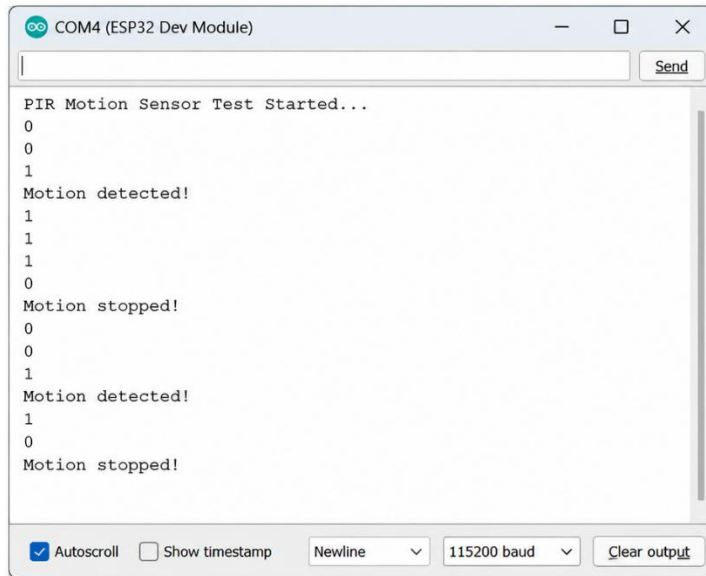


Figure 1: Serial Monitor Output

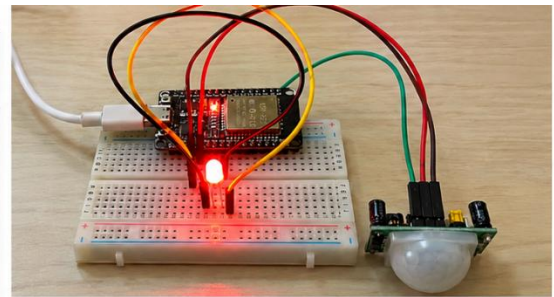


Figure 2: Motion Detected (LED ON)

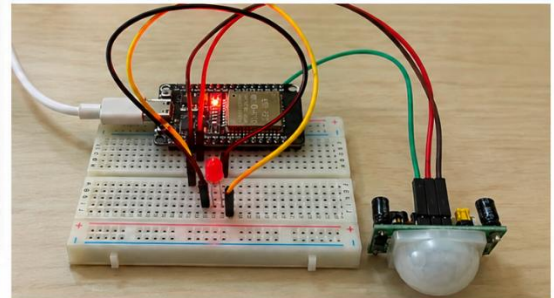


Figure 3: No Motion (LED OFF)

## Observations

7. When no person is detected, the PIR sensor output is LOW (0) and the LED remains OFF.
8. When an unauthorized person enters the sensing area, the PIR sensor output becomes HIGH (1), the LED turns ON, and "Motion detected!" is displayed.
9. When the detected motion stops, the LED turns OFF and "Motion stopped!" is displayed on the Serial Monitor.
10. The system can be used to monitor the presence of an unauthorized person in a restricted area.

## Conclusion

Thus, the ESP32 successfully detected the presence of a person using the PIR motion sensor and controlled the LED according to the detected motion. The Serial Monitor also displayed the corresponding motion status.